

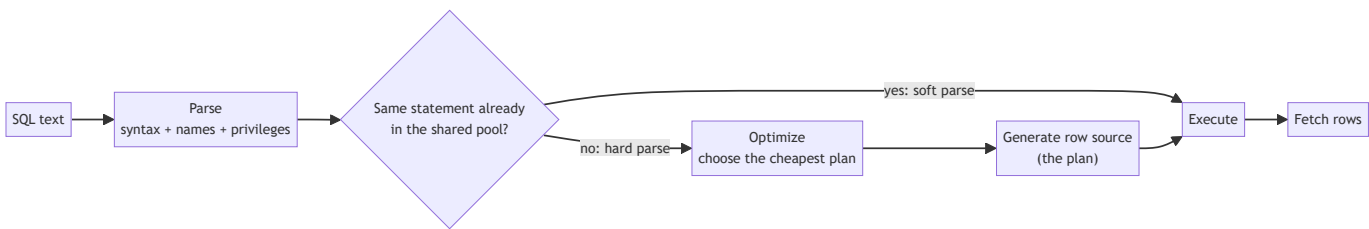
Extra 4 · Query Tuning and Execution Plans

Why this matters in interviews. "This query is slow — what do you do?", "How do you read an execution plan?", "Why isn't my index being used?", "Nested loop vs hash join?" Experienced-role interviews always reach performance. Chapter 10 taught what an index is; this chapter teaches how the database **chooses** a plan, how to **read** it, and the rewrites that fix the usual suspects.

In this chapter: how a query is processed · the optimizer and statistics · viewing and reading a plan · access paths · join methods · SARGable predicates (with executed plans) · rewrite recipes · bind variables · hints · a tuning method.

A note on the plans shown. The plans printed below were produced by **SQLite** (the engine used to run the examples in these notes) because Oracle isn't available where they were built. They demonstrate the *ideas* — scan vs index search, when an index is usable — which are the same in every database. Oracle's output looks different, so where Oracle's format matters an **illustrative** sample is given, clearly marked. Row counts and costs in real Oracle output will differ.

E4.1 What happens when you run a query



- **Parsing** checks that the SQL is valid and that the tables/columns exist.
- **Hard parse** = the expensive full optimization. **Soft parse** = Oracle finds an identical statement already parsed and reuses its plan. This is why **bind variables** matter (§E4.9).
- The **optimizer** picks a plan; the plan is a tree of operations (scan this, join with that...).

Oracle's optimizer is **cost-based (CBO)**: it estimates the cost of many possible plans using **statistics** about the data and picks the lowest. (The old rule-based optimizer is gone since 10g.)

E4.2 Statistics — the optimizer's eyes

The optimizer can't look at all the data every time; it relies on stored **statistics**:

Statistic	Used for
number of rows and blocks in a table	cost of a full scan
number of distinct values per column	selectivity — what fraction of rows a filter keeps
min / max / NULL count	range estimates
histograms	skewed data (99% of orders are 'DELIVERED' , 1% 'FAILED')

Statistic	Used for
index statistics (height, leaf blocks, clustering factor)	cost of an index scan

Selectivity = fraction of rows returned. **Cardinality** = estimated number of rows = selectivity × total rows.

```
-- refresh statistics after big data changes
BEGIN
    DBMS_STATS.GATHER_TABLE_STATS(ownname => 'HR', tabname => 'ORDERS');
END;
/
```

The #1 cause of a plan that suddenly gets slow is stale or missing statistics: the table grew tenfold, the optimizer still thinks it's tiny and picks a plan for a tiny table. Oracle gathers statistics automatically each night, but after a bulk load you should gather them yourself.

Index or full scan? It depends on selectivity. Reading 1% of a table through an index is fast; reading 60% through an index is *slower* than one sequential full scan, because each index hit is a random block read. So the optimizer *correctly* ignores an index when the filter isn't selective enough — "my index isn't used" is not always a bug.

E4.3 Viewing the plan

Tool	How
EXPLAIN PLAN FOR ... then DBMS_XPLAN.DISPLAY	the estimated plan, without running the query
SQL Developer	select the query and press F10 (Explain Plan)
SET AUTOTRACE ON (SQL*Plus / SQL Developer script)	runs the query and shows plan + statistics (consistent gets, physical reads)
DBMS_XPLAN.DISPLAY_CURSOR	the actual plan of a statement that already ran (ALLSTATS LAST shows real vs estimated rows)
SQL trace + TKPROF	detailed time breakdown per statement

```
EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE email = 'user500@mail.com';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Illustrative Oracle output (shape only — numbers omitted):

Id	Operation	Name	
0	SELECT STATEMENT		
1	TABLE ACCESS BY INDEX ROWID BATCHED	CUSTOMERS_BIG	
* 2	INDEX RANGE SCAN	IDX_CUSTOMERS_EMAIL	

Predicate Information:

2 - access("EMAIL"='user500@mail.com')

How to read a plan

- The plan is a **tree shown by indentation**. The most indented operation runs **first**; its result feeds the operation above it.
- Among siblings at the same level, they run **top to bottom** (the first child first).
- A ***** next to an Id means it has a filter/access condition listed under **Predicate Information**.
- Compare **estimated rows (E-Rows)** to **actual rows (A-Rows)**: a big mismatch means bad statistics — the classic cause of a bad plan.

E4.4 Access paths — how a table is read

Operation (Oracle name)	What it does	Good when
TABLE ACCESS FULL	reads every block of the table	you need a large share of the rows, or the table is small
INDEX UNIQUE SCAN	finds at most one row through a unique/PK index	<code>WHERE pk = :x</code>
INDEX RANGE SCAN	walks a range of index entries	<code>=</code> on a non-unique column, <code>BETWEEN</code> , <code>LIKE 'abc%'</code>
TABLE ACCESS BY INDEX ROWID	fetches the actual row using the ROWID the index gave	follows an index scan
INDEX FULL SCAN	reads the whole index in order	avoids a sort (<code>ORDER BY</code>)
INDEX FAST FULL SCAN	reads the whole index with multiblock reads, unordered	the index contains all the needed columns
INDEX SKIP SCAN	uses a composite index even without its leading column	leading column has very few distinct values

Covering index (index-only access): if an index contains *every column the query needs*, Oracle never touches the table at all — a large saving.

Oracle B-tree quirk: entirely- `NULL` keys are **not stored** in a single-column B-tree index, so `WHERE col IS NULL` **cannot** use that index (a composite index whose other column is non-null, or a function-based/constant trick, can).

E4.5 Join methods

When joining tables, the optimizer picks a **join method** and a **join order**.

Nested Loops

for each row in the outer table,
look up matches in the inner table

Hash Join

build a hash table from the smaller input,
probe it with the larger

Sort-Merge Join

sort both inputs on the join key,
merge them

Method	How it works	Best when	Notes
Nested loops	loop over the outer rows; each does an index lookup in the inner table	small outer set , indexed inner join column; first rows needed fast (OLTP)	cost $\approx$ outer rows $\times$ lookup cost
Hash join	hash the smaller table in memory, scan the bigger one and probe	large inputs, no useful index, equality join (=)	needs memory; the workhorse of reports
Sort-merge	sort both by the join key and merge	inputs already sorted, or non-equality joins (< , > , BETWEEN)	hash join can't do inequalities

You can nudge the optimizer with hints (`USE_NL` , `USE_HASH` , `USE_MERGE` , `LEADING`) — see §E4.10 — but the right fix is usually **statistics and indexes**.

The trap of the cartesian join. A missing join condition shows in the plan as `MERGE JOIN CARTESIAN` — every row $\times$ every row. If a plan contains it unintentionally, someone forgot an `ON` .

E4.6 SARGable predicates — writing filters an index can use

SARG = *Search ARGument-able*. A predicate is SARGable if the database can use an index to jump to the matching rows. The most common tuning mistake is wrapping the **indexed column** in something, so the index becomes useless.

We use a table `customers_big` with 2,000 rows and `cust_id` as primary key.

Without an index — the whole table is scanned

```
EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE email = 'user500@mail.com';
```

Result

Operation (SQLite plan)
SCAN customers_big

SCAN = full table scan: every one of the 2,000 rows is examined.

With an index — a direct lookup

```
CREATE INDEX idx_customers_email ON customers_big (email);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE email = 'user500@mail.com';
```

Result

Operation (SQLite plan)
SEARCH customers_big USING INDEX idx_customers_email (email=?)

SEARCH ... USING INDEX = jump straight to the row. (In Oracle: INDEX RANGE SCAN + TABLE ACCESS BY INDEX ROWID .)

A function on the column breaks the index

```
CREATE INDEX idx_customers_email ON customers_big (email);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE UPPER(email) = 'USER500@MAIL.COM';
```

Result

Operation (SQLite plan)
SCAN customers_big

The index is sorted by `email`, not by `UPPER(email)`, so the database has no choice but to compute `UPPER` for every row: back to a **full scan**.

Fix 1 — compare the column directly (store data in a consistent case). **Fix 2 — a function-based index** on the exact expression:

```
CREATE INDEX idx_customers_upper_email ON customers_big (UPPER(email));

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE UPPER(email) = 'USER500@MAIL.COM';
```

Result

Operation (SQLite plan)
SEARCH customers_big USING INDEX idx_customers_upper_email (=?)

LIKE : a leading wildcard kills the index, a trailing one doesn't

```
CREATE INDEX idx_customers_email ON customers_big (email);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE email LIKE 'user50%';
```

Result

Operation (SQLite plan)

SEARCH customers_big USING INDEX idx_customers_email (email>? AND email<?)

'user50%' has a known **prefix**, so the index can find the range that starts with user50 .

```
CREATE INDEX idx_customers_email ON customers_big (email);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE email LIKE '%50@mail.com';
```

Result

Operation (SQLite plan)

SCAN customers_big

'%50@mail.com' could match anywhere, so every entry must be checked: **full scan**. (For real "contains" searches use Oracle Text or a reverse-key/function-based approach.)

Composite index — the leading column rule

```
CREATE INDEX idx_city_signup ON customers_big (city, signup);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE city = 'Pune' AND signup = '2024-03-01';
```

Result

Operation (SQLite plan)

SEARCH customers_big USING INDEX idx_city_signup (city=? AND signup=?)

Both columns are used (city=? AND signup=?). But filter on the **second** column alone:

```
CREATE INDEX idx_city_signup ON customers_big (city, signup);

EXPLAIN PLAN FOR
SELECT * FROM customers_big WHERE signup = '2024-03-01';
```

Result

Operation (SQLite plan)
SCAN customers_big

The index is sorted by `city` first, so it is useless when `city` isn't in the filter (Oracle may still choose an **INDEX SKIP SCAN** here, because `city` has only 4 distinct values — a special case, not something to rely on). **Put the column you filter with most often, and with equality, first.**

Covering index — answered from the index alone

```
CREATE INDEX idx_city_signup ON customers_big (city, signup);

EXPLAIN PLAN FOR
SELECT city, signup FROM customers_big WHERE city = 'Pune';
```

Result

Operation (SQLite plan)
SEARCH customers_big USING COVERING INDEX idx_city_signup (city=?)

COVERING INDEX — both requested columns are inside the index, so the table is never read.

Summary: SARGable or not?

Predicate	Index usable?	Fix
WHERE email = :x	✓	–
WHERE UPPER(email) = :x	✗	function-based index, or store normalized
WHERE TRUNC(created) = DATE '2025-08-16'	✗	created >= DATE '2025-08-16' AND created < DATE '2025-08-17'
WHERE salary * 12 > 600000	✗	salary > 600000 / 12
WHERE email LIKE 'user5%'	✓	–
WHERE email LIKE '%5@mail.com'	✗	Oracle Text / rethink
WHERE phone = 9876543210 (phone is VARCHAR2)	✗ — Oracle converts the column : TO_NUMBER(phone)	compare with a string: phone = '9876543210'
WHERE col IS NULL (single-column B-tree)	✗ (NULLs aren't indexed)	composite index with a NOT NULL column, or a sentinel
WHERE status <> 'X' / NOT IN	usually ✗	rewrite as a positive condition if possible
WHERE a = :x OR b = :y	often ✗	UNION ALL of two indexed queries

The implicit conversion trap deserves special mention: comparing a `VARCHAR2` column to a *number* forces Oracle to convert the column side. It silently defeats the index — and if any non-numeric string is in the column, it can raise **ORA-01722 invalid number** at runtime.

E4.7 Rewrite recipes — same answer, cheaper plan

The same question can be written many ways; the results are identical, the cost is not.

Three ways to ask "who earns more than their department's average?"

```
SELECT e.emp_name, e.salary
FROM   emp e
WHERE  e.salary > (SELECT AVG(salary) FROM emp WHERE dept_id = e.dept_id)
ORDER BY e.emp_name;
```

Result

emp_name	salary
Bala	70000
Chitra	50000
Dinesh	50000
Farhan	65000

A **correlated subquery** — conceptually re-runs the inner query for every employee. Same result with a **join to a pre-aggregated table** (aggregate once):

```
SELECT e.emp_name, e.salary
FROM   emp e
JOIN   (SELECT dept_id, AVG(salary) AS avg_sal FROM emp GROUP BY dept_id) d
      ON d.dept_id = e.dept_id
WHERE  e.salary > d.avg_sal
ORDER BY e.emp_name;
```

Result

emp_name	salary
Bala	70000
Chitra	50000
Dinesh	50000
Farhan	65000

And with a **window function** — one pass over the table:

```
SELECT emp_name, salary
FROM   (SELECT emp_name, salary,
              AVG(salary) OVER (PARTITION BY dept_id) AS avg_sal
        FROM   emp)
WHERE  salary > avg_sal
ORDER BY emp_name;
```

Result

emp_name	salary
Bala	70000
Chitra	50000
Dinesh	50000
Farhan	65000

All three return **Bala, Chitra, Dinesh and Farhan**. (Modern optimizers often *unnest* the first form into the second automatically — measure before assuming one is slower.)

The checklist of common rewrites

Slow pattern	Better	Why
<code>SELECT *</code>	list only needed columns	less I/O, enables covering indexes
<code>WHERE FUNC(col) = ...</code>	move the function to the constant side / function-based index	keeps the index usable
<code>HAVING dept_id = 20</code> (a non-aggregate condition)	<code>WHERE dept_id = 20</code>	filters before grouping — far fewer rows to group
<code>UNION</code> when duplicates can't occur	<code>UNION ALL</code>	<code>UNION</code> does an extra sort/hash to remove duplicates
<code>NOT IN (subquery)</code>	<code>NOT EXISTS</code> (or <code>LEFT JOIN ... IS NULL</code>)	NULL-safe, and usually a better anti-join plan
<code>IN (huge list) / IN (big subquery)</code>	<code>EXISTS</code> or a join	lets the optimizer stop at the first match
<code>DISTINCT</code> to hide duplicate join rows	fix the join condition	a symptom, not a cure — extra sort
<code>COUNT(*)</code> on a huge table just to test existence	<code>EXISTS</code> / <code>FETCH FIRST 1 ROW ONLY</code>	stops at first row
<code>OR</code> across different columns	<code>UNION ALL</code> of separate indexed queries	each branch can use its own index
Row-by-row loop in PL/SQL (<code>FOR r IN ... LOOP UPDATE ...</code>)	one set-based <code>UPDATE</code> / <code>MERGE</code> , or <code>FORALL</code>	avoids thousands of SQL↔PL/SQL context switches
<code>ORDER BY</code> with no need	remove it	sorting is expensive
Deep pagination <code>OFFSET 100000 ROWS</code>	keyset: <code>WHERE id > :last_id ORDER BY id</code> <code>FETCH FIRST 20 ROWS ONLY</code>	offset still reads and throws away the skipped rows
Commit after every row of a bulk load	commit in batches	fewer log waits

Set-based vs row-by-row

The single biggest performance rule for procedural code: **one SQL statement that processes many rows beats a loop that runs one statement per row.**

```

-- slow: 10,000 executions of the UPDATE, 10,000 context switches
FOR r IN (SELECT emp_id FROM emp WHERE dept_id = 20) LOOP
    UPDATE emp SET salary = salary * 1.1 WHERE emp_id = r.emp_id;
END LOOP;

-- fast: one statement
UPDATE emp SET salary = salary * 1.1 WHERE dept_id = 20;

```

E4.8 Indexes have a price

An index speeds up reads but slows every `INSERT`, `UPDATE` (of indexed columns) and `DELETE`, and uses disk space.

Index it	Don't bother
foreign key columns (also avoids Oracle locking trouble on the parent)	tiny tables
columns used in frequent <code>WHERE</code> / <code>JOIN</code> / <code>ORDER BY</code>	columns with few distinct values on big tables (a plain B-tree; bitmap indexes suit these in reporting systems, never in busy OLTP)
highly selective columns	columns updated constantly
	tables loaded in bulk and rarely queried

Rules of thumb: **fewer, well-chosen composite indexes** beat many single-column ones; drop indexes that no query uses.

E4.9 Bind variables

Two statements that differ only in a literal are, to Oracle, **different statements**:

```

SELECT * FROM orders WHERE customer_id = 101;    -- hard parse
SELECT * FROM orders WHERE customer_id = 102;    -- another hard parse

```

With a **bind variable** the text is identical, so the plan is parsed once and reused:

```

SELECT * FROM orders WHERE customer_id = :cust_id;    -- one parse, many executions

```

Benefits: **far less parsing** (faster, less shared-pool pressure) and **protection against SQL injection** — the value is never treated as SQL text. In PL/SQL, variables used in static SQL are bound automatically; in dynamic SQL use `EXECUTE IMMEDIATE '...' USING x` rather than concatenating.

Caveat — bind peeking. Oracle looks at the first bind value when it builds a plan. With very skewed data, one plan may not suit every value; adaptive cursor sharing and histograms mitigate this.

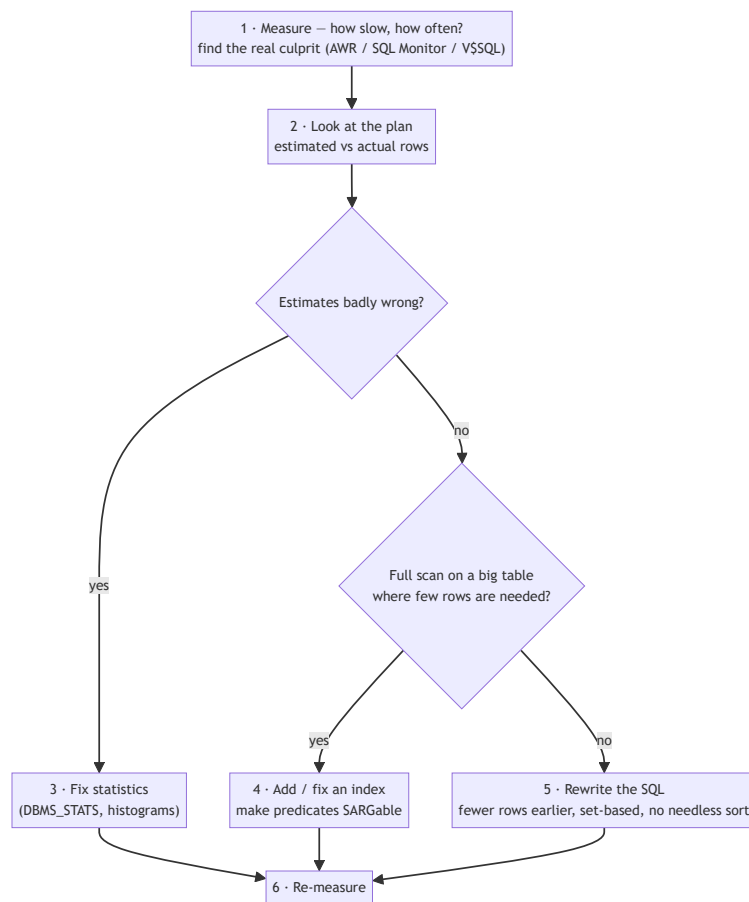
E4.10 Hints — a last resort

A **hint** is a comment that tells the optimizer what to do:

```
SELECT /*+ INDEX(e idx_emp_dept) */ * FROM emp e WHERE dept_id = 20;  
SELECT /*+ FULL(e) */ * FROM emp e WHERE dept_id = 20;  
SELECT /*+ LEADING(d e) USE_HASH(e) */ e.emp_name, d.dept_name  
FROM dept d JOIN emp e ON e.dept_id = d.dept_id;  
SELECT /*+ FIRST_ROWS(10) */ * FROM orders ORDER BY created DESC;
```

- Hints are **suggestions**; a malformed hint is silently ignored (no error).
- They freeze a decision that may be wrong as data grows. **Fix statistics and indexes first**; use hints for a targeted, documented workaround.
- Remember the alias: `INDEX(e ...)` refers to the alias `e`, not the table name.

E4.11 A method for "the query is slow"



1. **Measure first.** Don't guess. Find the statement that consumes the most time (AWR/ASH reports, `V$SQL`, SQL Monitor, `AUTOTRACE`).
2. **Read the plan.** Look for full scans of big tables, huge row-count estimates, `MERGE JOIN CARTESIAN`, and estimates far from actuals.
3. **Check statistics.** Gather if stale.
4. **Check indexes and predicates.** Is a function/conversion hiding the column?

5. **Reduce work early.** Filter before joining, select fewer columns, aggregate before joining.
6. **Think set-based**, then consider partitioning, materialized views or caching for the truly heavy cases.
7. **Re-measure**, and change one thing at a time.

Useful numbers from AUTOTRACE: *consistent gets* (logical block reads — the best measure of work), *physical reads* (disk), *sorts (disk)* (should be zero).

Interview corner

Q. A query is slow. How do you approach it? Measure it, get the execution plan, compare estimated with actual rows, check statistics, indexes and predicates (SARGability), then rewrite (fewer rows early, set-based) and re-measure. Hints last.

Q. What is an execution plan? The tree of operations the optimizer chose to run a statement — access paths, join methods and join order — shown with `EXPLAIN PLAN / DBMS_XPLAN`.

Q. Full table scan vs index scan — which is faster? Neither always. An index wins when few rows are needed; a full scan wins when a large fraction is needed or the table is small. The optimizer decides by cost.

Q. Why might an index not be used? A function or implicit conversion on the column, a leading wildcard, `NULL` conditions, the query isn't selective enough, the leading column of a composite index is missing, or stale statistics.

Q. Nested loops vs hash join vs sort-merge? Nested loops: small outer set, indexed inner lookups. Hash: big unsorted inputs, equality joins. Sort-merge: pre-sorted inputs or non-equality joins.

Q. What are optimizer statistics and why do they matter? Data-distribution figures the cost-based optimizer uses to estimate cardinality. Stale statistics produce wrong estimates and bad plans; refresh with `DBMS_STATS`.

Q. What is a covering index? An index containing every column a query needs, so the table isn't read.

Q. Why use bind variables? Identical statement text lets Oracle reuse the parsed plan (soft parse) and prevents SQL injection.

Q. WHERE vs HAVING for performance? Put every non-aggregate condition in `WHERE` so rows are discarded before grouping.

Q. EXISTS vs IN ? Often the same plan in modern Oracle. `EXISTS` is NULL-safe and stops at the first match; `NOT EXISTS` beats `NOT IN` because of NULLs.

Q. What does "SARGable" mean? A predicate written so an index can be used — no function/arithmetic on the indexed column, no leading wildcard, no implicit type conversion.

Quick recap

- The **cost-based optimizer** picks the cheapest plan using **statistics**; stale stats ⇒ bad plans.
- See plans with `EXPLAIN PLAN + DBMS_XPLAN.DISPLAY`, `AUTOTRACE` or F10; read **innermost first**; compare estimated vs actual rows.
- Access paths: full scan · unique/range index scan · fast full scan · skip scan; **covering** index avoids the table.
- Joins: **nested loops** (small outer + index) · **hash** (large, equality) · **sort-merge** (sorted / non-equality).
- Keep predicates **SARGable**: no function on the column, no leading %, no implicit conversion, mind `IS NULL` and composite-index order.

- Rewrites: `WHERE` not `HAVING`, `UNION ALL`, `NOT EXISTS`, no `SELECT *`, set-based over row-by-row, keyset pagination.
- **Bind variables** for parse reuse and security; hints are a last resort; **measure** → **plan** → **stats** → **index** → **rewrite** → **re-measure**.